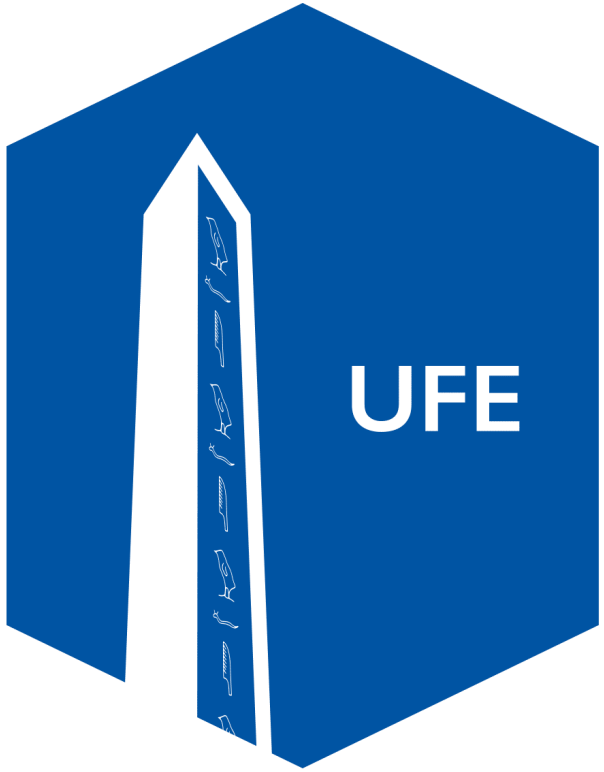

UFE

LearnAI

From Lesson PDFs to Playable 3D Missions

Made by

1. Seif Ramy
2. Amr Mohmed Talaat

PROJECT SCOPE

What we’re building

A lesson PDF becomes a short mission in a reusable 3D level.

Upload a lesson	The student selects a text-based PDF about electric circuits.
Restore power	Walk through a 3D lab and use three consoles. At one console, choose the circuit that will light a bulb.
Complete the mission	Correct answers activate the consoles. The exit opens when all three are complete.
What changes per PDF	The AI generates the story, challenges and feedback. The level, controls and exit rules are shared.

METHODOLOGY

Prototype, generate, verify

Build a playable example first, then check the generated missions in two ways.

01 Build by hand	Make the lab playable with a mission written by the team.
02 Generate content	Give a language model the lesson text and a fixed format for the mission, challenges and source evidence.
03 Evaluate separately	Play through the mission in a browser. Then check each answer and explanation against its cited PDF page.

OpenGame influence Figure 2(a) trains a code model in advance. LearnAI uses an existing model and borrows the reusable template and browser checks.

METHODOLOGY

Prototype, generate, verify

Build a playable example first, then check the generated missions in two ways.

01 Build by hand	Make the lab playable with a mission written by the team.
02 Generate content	Give a language model the lesson text and a fixed format for the mission, challenges and source evidence.
03 Evaluate separately	Play through the mission in a browser. Then check each answer and explanation against its cited PDF page.

OpenGame influence Figure 2(a) trains a code model in advance. LearnAI uses an existing model and borrows the reusable template and browser checks.

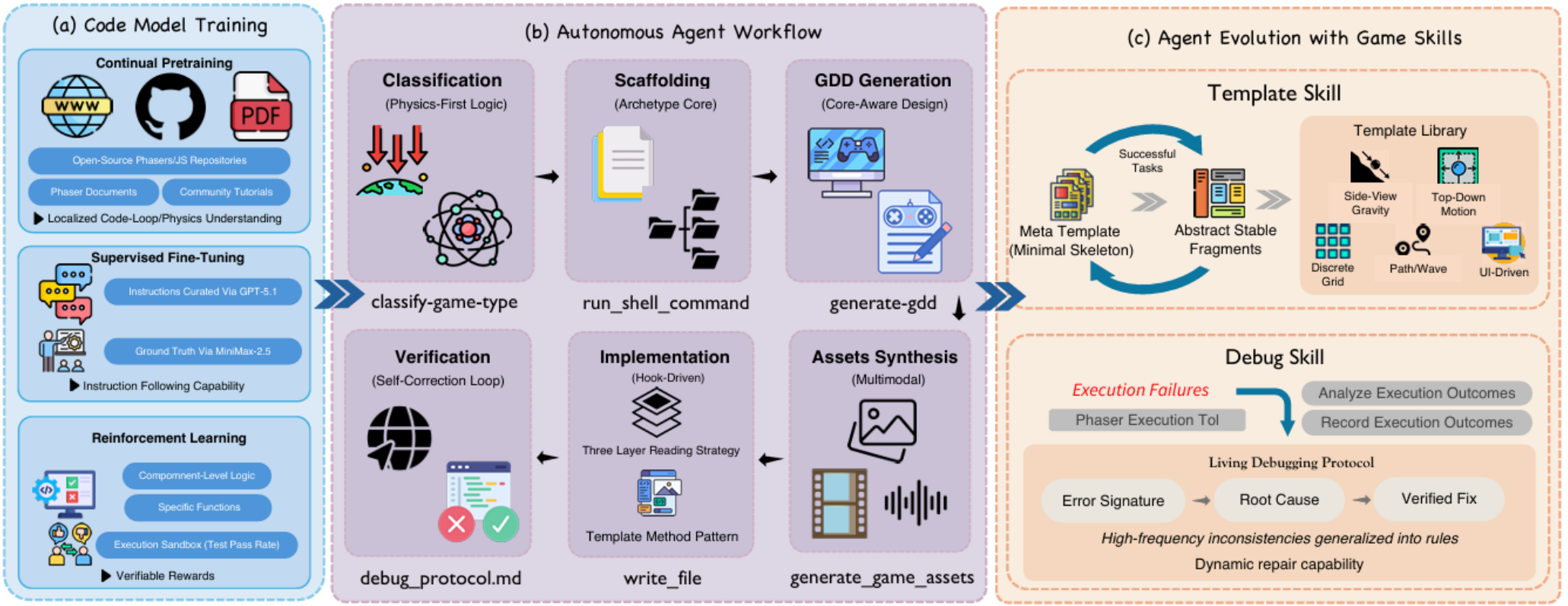
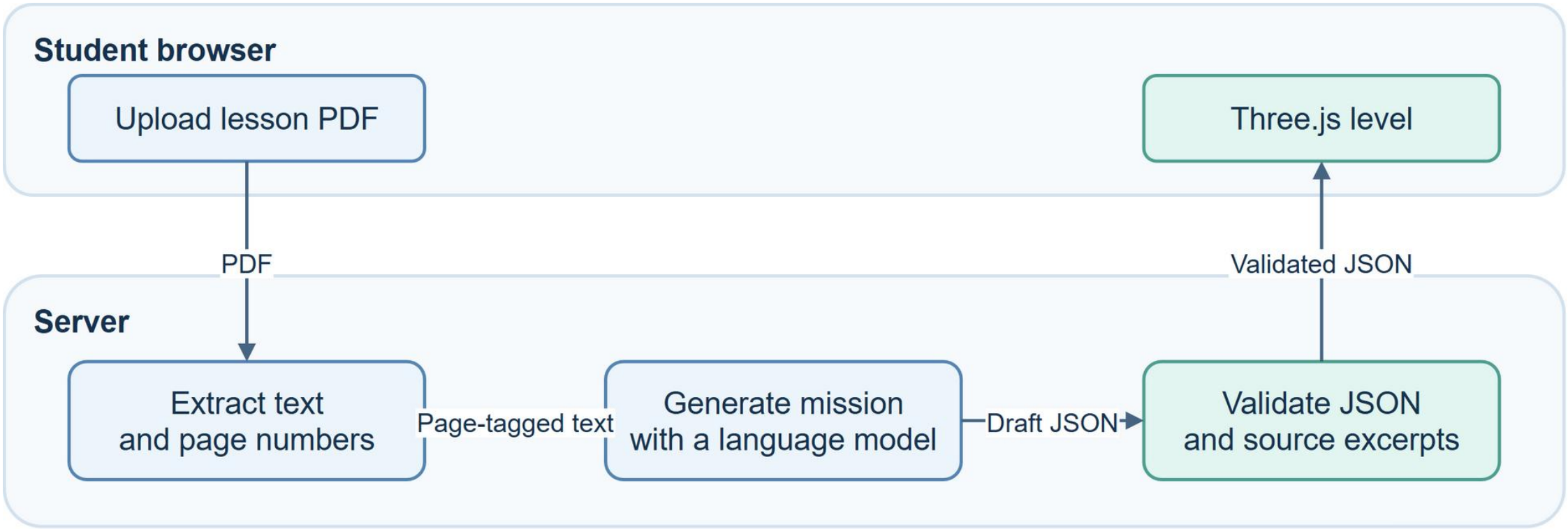


Figure 2: **The OpenGame architecture.** The framework integrates three coupled components: (a) a multi-stage code-model training pipeline that establishes engine-specific priors, (b) an autonomous agent workflow that translates natural-language game ideas into runnable projects through a structured six-phase process, and (c) an agent-evolution module that continuously refines structural scaffolding (Template Skill) and repair behavior (Debug Skill) through accumulated experience.

LearnAI architecture

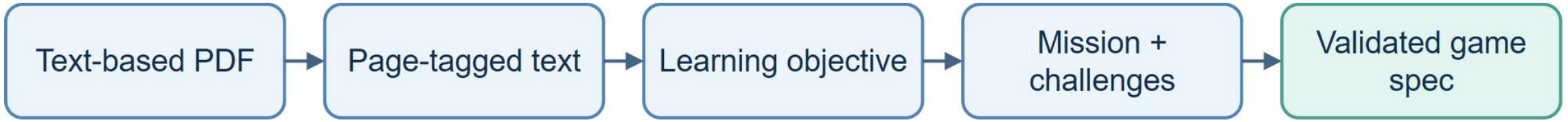
The server turns a lesson into a validated game specification. Three.js plays it in the browser.



DATA FLOW

PDF-to-game pipeline

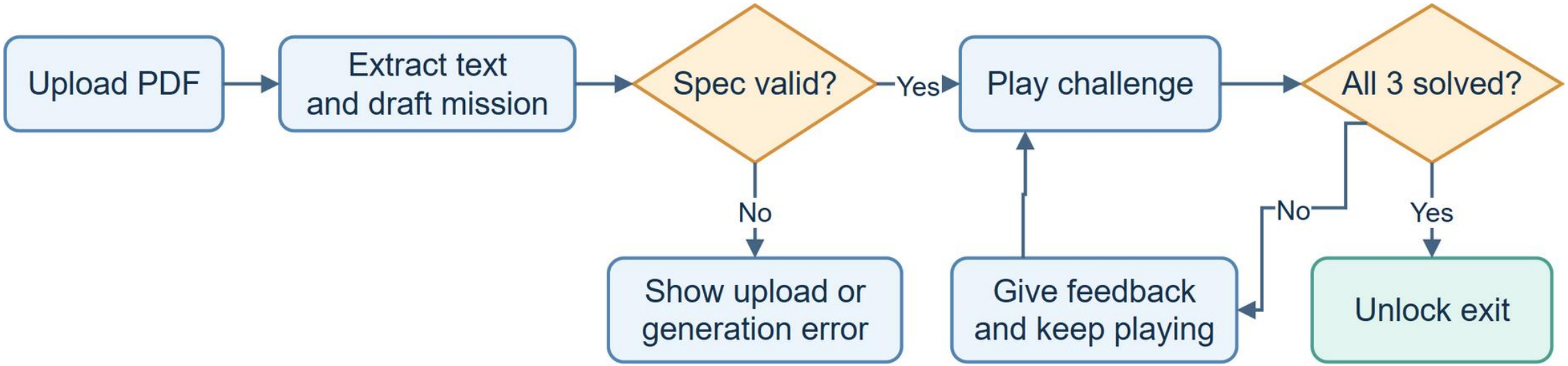
Follow the lesson data from the uploaded document to the mission loaded by the game.



Objective and mission	Identify a closed circuit. Restore power to the lab.
Challenge and answer	Which circuit lights the bulb? Choose the circuit with a closed path.
Feedback and evidence	“A closed path lets current flow.” Illustrative source page: 2.

Activity workflow

The activity diagram follows the upload, the validation decision and the gameplay loop.



A wrong answer gives feedback. The exit unlocks after all three stations are solved.

BUILD ORDER

How we'll start

The first playable milestone is one console, one correct answer and one door.

01 One-console demo	Build the Three.js room, movement, one hand-written challenge and an exit that opens after a correct answer.
02 Connect one PDF	Extract page-tagged text. Generate and validate a mission specification, then load it into the room.
03 Expand and test	Add the other two stations. Generate missions from two PDFs and test a full playthrough for each.

SEMESTER TARGET

Deliverables and acceptance

Demonstrate the same 3D level with two missions generated from different lesson PDFs.

Playable browser demo	Upload each PDF and complete its three challenges. Show feedback on a wrong answer and the exit opening after the third correct answer.
Source code and setup	Hand over the project repository, a setup guide, the two sample PDFs and the mission specification format.
Short evaluation	Record whether both missions load and can be completed. Check all six answers and explanations against their cited source pages.

IMPLEMENTATION STACK

Technologies and GitHub

Each tool has a specific role in the semester prototype.

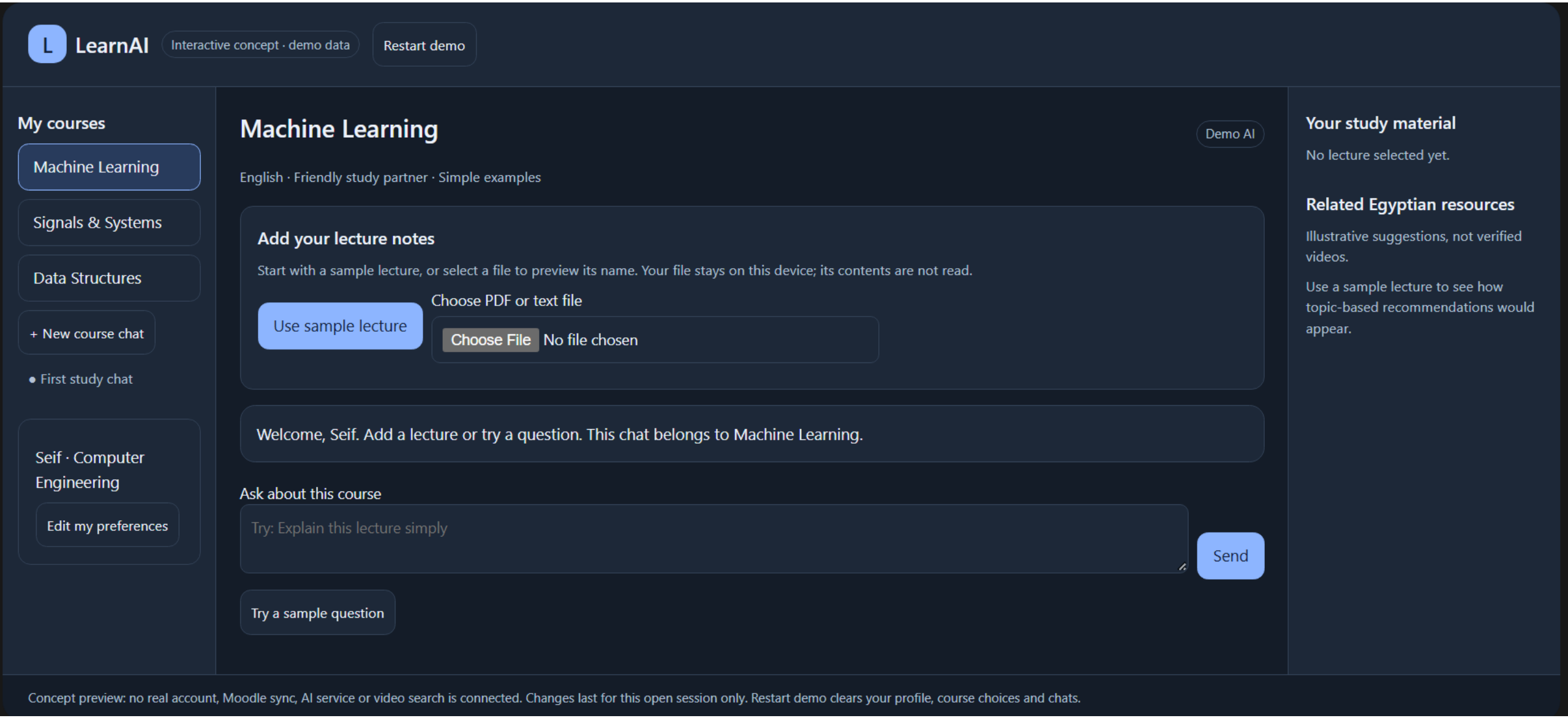
Three.js + TypeScript	Render the 3D lab and implement movement, interactions and game state.
PDF.js + Node.js	Extract page-tagged text. Run the model call and validate its response on the server.
Language model + JSON Schema	Generate the lesson mission as JSON and check its required fields and allowed values.
Playwright + draw.io	Exercise the browser gameplay and maintain the architecture, pipeline and activity diagrams.

Papers and project links

Start with OpenGame Figure 2 and §3 before reading the rest of the paper.

OpenGame paper	Reusable templates and execution checks for generated web games.
OpenGame on GitHub	Reference implementation and playable demos.
Retrieval-Augmented Generation	Background reading on grounding generation in external text.
Implementation repositories	Three.js | PDF.js | Playwright | draw.io

First Draft for our technology



LearnAI

↺ Reset bench

APPLIED MACHINE LEARNING / FIELD LAB

The Neural Workshop

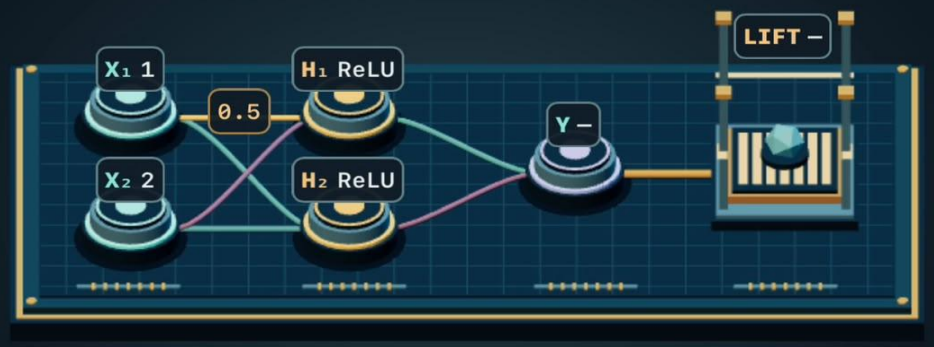
Send a signal through a tiny neural network and use its output to power a lift.

MISSION OBJECTIVE

Lift target ≥ 1.50

Predict the output to complete the challenge.

WORKBENCH 01 / FORWARD PASS NETWORK 02 / 02 / 01



Positive signal Negative signal Selected connection

01 / ADJUST

Tune the circuit

Component

$X_1 \rightarrow H_1$

Weight

0.5

Try 3 to see what changes at the first ReLU.

02 / TEST

Predict & send

Predicted output y

e.g. 0.5

Send signal ↗

Predict the output, then send a signal through the network.

LEARN AI / EXPERIMENT 001 2 INPUTS · 2 ReLU UNITS · 1 LINEAR OUTPUT